

Multi-step workflows

LESSON 19 · MODULE 4 — TOOLS & WORKFLOWS

What you'll learn

- How to build a multi-agent system the **built-in way** — one manager, several specialists
- How to register workers with **managed_agents** and a usage description that tells the manager when to use each
- How the manager **delegates and synthesizes in a single call** — you describe the goal, not the steps
- How this differs from the **manual orchestration** you wrote earlier in the series

Key concepts

You've orchestrated by hand twice; this is the built-in way. Earlier you wired multi-agent flows yourself in Python — the classify-then-answer chatbot (lesson 6) and the plan-then-produce creative assistant (lesson 9). Both times *you* scripted the order. Now: a manager agent. One manager, several specialists, and the manager delegates and synthesizes — all from a single call.

Specialists are just normal agents. Build them first with the same **create_agent** you've used the whole series — one researcher, one writer, each good at a single job. On their own they don't know about each other. The manager is what wires them together.

managed_agents registers the workers; the usage description is the routing hint. The manager takes one new thing: a list of workers, each registered with its id, a name, and a **usage description** that tells the manager *when* to reach for it. That description is the routing hint — the manager reads it to decide which worker fits the task. That's the whole setup.

You describe the goal; the manager decides the steps. You're not scripting the order anymore. Give the manager the goal in one **run**, and it decides which specialist to call, in what order, and how to combine what comes back. Under the hood it ran the researcher, handed the findings to the writer, and synthesized the result — all from that single **manager.run**. You never told it the order; it read the two usage descriptions and routed the work itself.

That's the difference from manual orchestration. In lessons 6 and 9 you owned the control flow. Here the manager owns it — you own the *goal* and the *routing hints*. One call in, a finished result out.

TAKEAWAY

You've orchestrated by hand twice; this is the built-in way.

TAKEAWAY

Specialists are just normal agents.

TAKEAWAY

You describe the goal; the manager decides the steps.

TAKEAWAY

That's the difference from manual orchestration.

Code shown in this lesson

PYTHON

```
from lyzr import Studio

studio = Studio(api_key="...")

# Build the specialists first – each is a normal agent, same create_agent as always.
researcher = studio.create_agent(
    name="researcher", provider="gpt-4o",
    role="research analyst",
    goal="gather and summarize the key facts on a topic",
)
writer = studio.create_agent(
    name="writer", provider="gpt-4o",
    role="content writer",
    goal="turn research findings into a clear, written brief",
)
```

PYTHON

```
# The manager – one new parameter, managed_agents. Register each worker by id,
# with a name and a usage description that tells the manager when to use it.
manager = studio.create_agent(
    name="manager", provider="gpt-4o",
    role="project manager",
    goal="produce a finished brief on the requested topic",
    managed_agents=[
        {"id": researcher.id, "name": "researcher",
         "usage_description": "Use to gather and summarize facts on a topic."},
        {"id": writer.id, "name": "writer",
         "usage_description": "Use to turn research findings into a written brief."},
    ],
)

# One call: describe the goal and let the manager route the work.
# It runs the researcher, hands findings to the writer, and synthesizes the result.
result = manager.run("Write a short brief on the state of solid-state batteries.")
```

TRY THIS

- 1 Run the manager with only the goal — never naming the researcher or the writer — and confirm it calls both in the right order. The routing came entirely from the usage descriptions.
- 2 Make the writer's usage description vague ("does writing things") and rerun. See whether the manager still routes correctly, then restore a crisp description — proof the usage description is the routing interface.
- 3 Add a third specialist (e.g. an **editor** that tightens prose) and a usage description for it. Give the manager a goal that needs all three and watch it sequence research → write → edit on its own.